

WEBX Module 6

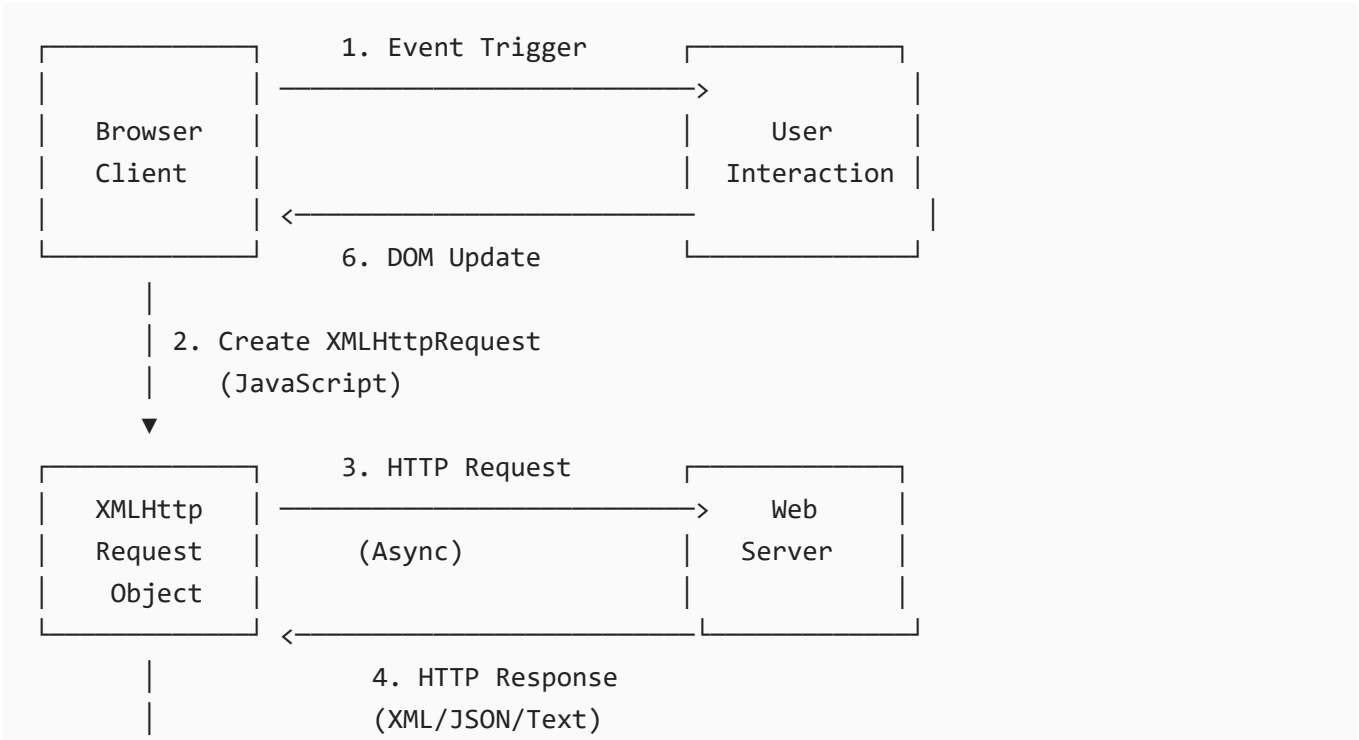
Explain Features and Working of AJAX with Diagram

AJAX (Asynchronous JavaScript and XML) is a set of web development techniques that allows web pages to communicate with servers asynchronously without reloading the entire page.

Features:

Feature	Description
Asynchronous Requests	Sends/receives data in background without blocking user interaction
Partial Page Updates	Updates only specific DOM elements instead of full page reload
Reduced Bandwidth	Only sends/receives necessary data, not entire HTML
Better User Experience	Smooth, responsive interfaces like desktop apps
Platform Independent	Works across all modern browsers and OS
Real-time Validation	Live form validation (e.g., username availability)
XML/JSON Support	Handles multiple data formats
No Plugins Required	Built on standard browser APIs

Diagram



5. Process Response
(Callback Function)



DOM
Update
(Display)

readyState Values:

Value	State	Description
0	UNSENT	XMLHttpRequest object created
1	OPENED	open() method called
2	HEADERS_RECEIVED	send() called, headers + status received
3	LOADING	Downloading response (partial data)
4	DONE	Operation complete

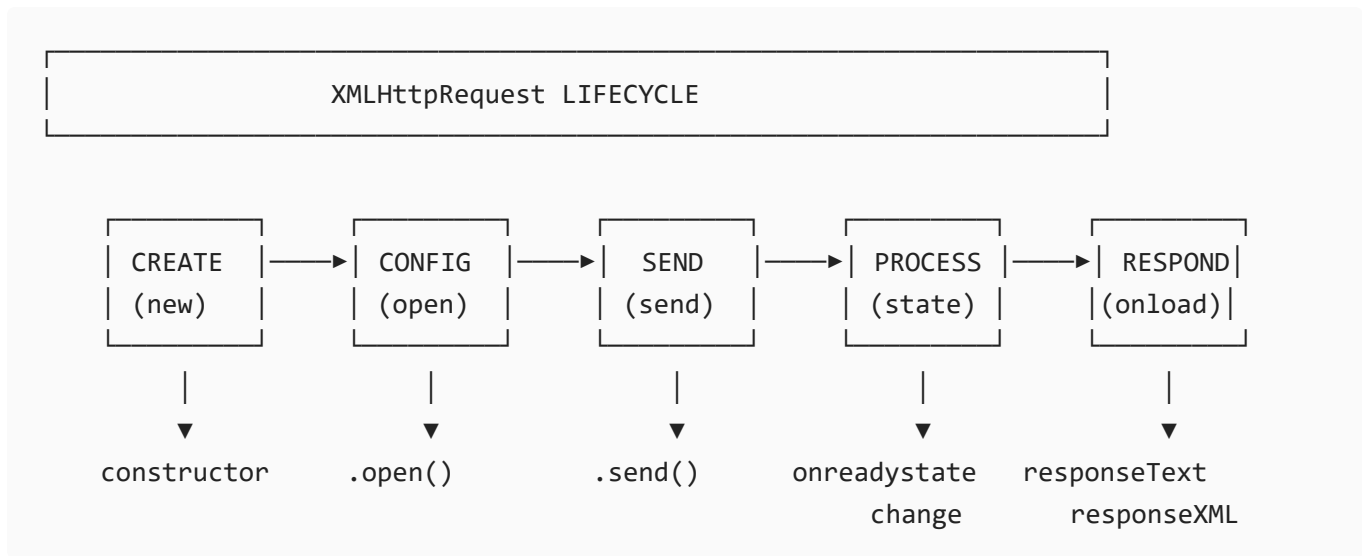
Example:

```
// Modern AJAX using Fetch API (simpler)
async function loadData() {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    document.getElementById('output').innerHTML = data.message;
  } catch(error) {
    console.error('Error:', error);
  }
}
```

How XMLHttpRequest Object Works in AJAX?

The **XMLHttpRequest** object is a browser API that enables JavaScript to make HTTP requests to servers without reloading the page. It's the foundation of AJAX technology.

Object Lifecycle:



Properties of XMLHttpRequest

Property	Description
readyState	Current state (0-4) of request
status	HTTP status code (200, 404, 500, etc.)
statusText	HTTP status message ("OK", "Not Found")
responseText	Response as text string
responseXML	Response as XML Document object
responseType	Expected response type (json, text, blob, etc.)
timeout	Timeout value in milliseconds
withCredentials	Send cookies with cross-origin requests

Methods:

Method	Purpose
open(method, url, async)	Initializes request (GET/POST, URL, sync/async)
send(data)	Sends request to server
setRequestHeader(header, value)	Adds HTTP header to request
getResponseHeader(header)	Returns specific response header
getAllResponseHeaders()	Returns all response headers
abort()	Cancels current request
overrideMimeType(type)	Overrides server's MIME type

Events:

Event	Triggers When...
onreadystatechange	readyState changes (most important)
onload	Request completes successfully
onerror	Request fails
onprogress	Data is being downloaded (for large files)
ontimeout	Request exceeds timeout value
onloadstart	Request begins
onloadend	Request ends (success or failure)

Discuss Technologies AJAX Works With

1. Client-Side Technologies:

Technology	Role in AJAX	Example
HTML/XHTML	Provides structure & elements to update	<code><div id="result"></div></code>
CSS	Styles the dynamically loaded content	Dynamic class changes
JavaScript	Controls the entire AJAX process	Event handlers, callbacks
DOM	Manipulates page content dynamically	<code>document.getElementById()</code>
XMLHttpRequest	Core transport mechanism	<code>new XMLHttpRequest()</code>
Fetch API	Modern alternative to XHR	<code>fetch(url).then()</code>

2. JavaScript Libraries & Frameworks:

Library/Framework	AJAX Features
jQuery	<code>\$.ajax()</code> , <code>\$.get()</code> , <code>\$.post()</code> , <code>\$.getJSON()</code>
Axios	Promise-based HTTP client for browser/Node.js
Angular	<code>HttpClient</code> module with observables
React	<code>fetch()</code> inside <code>useEffect</code> or libraries like <code>axios</code>
Vue.js	<code>axios</code> or native <code>fetch()</code>
Next.js	Server-side and client-side fetching

jQuery Example:

```
$.ajax({
  url: 'api/data',
  method: 'GET',
  success: function(response) {
    $('result').html(response);
  },
  error: function(xhr, status, error) {
    console.error(error);
  }
});
```

3. Server-Side Technologies:

Technology	How it Handles AJAX
PHP	<code>\$_GET['param'], \$_POST, json_encode(\$data)</code>
Node.js (Express)	<code>app.get('/api', (req,res) => res.json(data))</code>
Python (Django)	JsonResponse, DRF serializers
Python (Flask)	<code>jsonify()</code> response
Java (Servlets)	<code>response.setContentType("application/json")</code>
Ruby on Rails	<code>render json: @data</code>
ASP.NET Core	<code>return Ok(object) / return Json()</code>

PHP Example:

```
<?php
// api.php
header('Content-Type: application/json');
$data = ['name' => 'John', 'age' => 30];
echo json_encode($data);
?>
```

Node.js/Express Example:

```
app.get('/api/users', (req, res) => {
  const users = [{id:1, name:'John'}];
  res.json(users);
});
```

4. Data Formats:

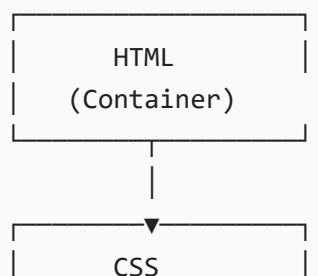
Format	Characteristics	Use Case
JSON	Lightweight, native to JS, easy to parse	✅ Most common (90%+ of AJAX)
XML	Heavier, verbose, requires XML parsing	Legacy systems, SOAP APIs
HTML	Direct insertion into DOM	Partial page fragments
Plain Text	Simple, no structure	Status messages, logs
FormData	Key-value pairs for file uploads	Forms with files

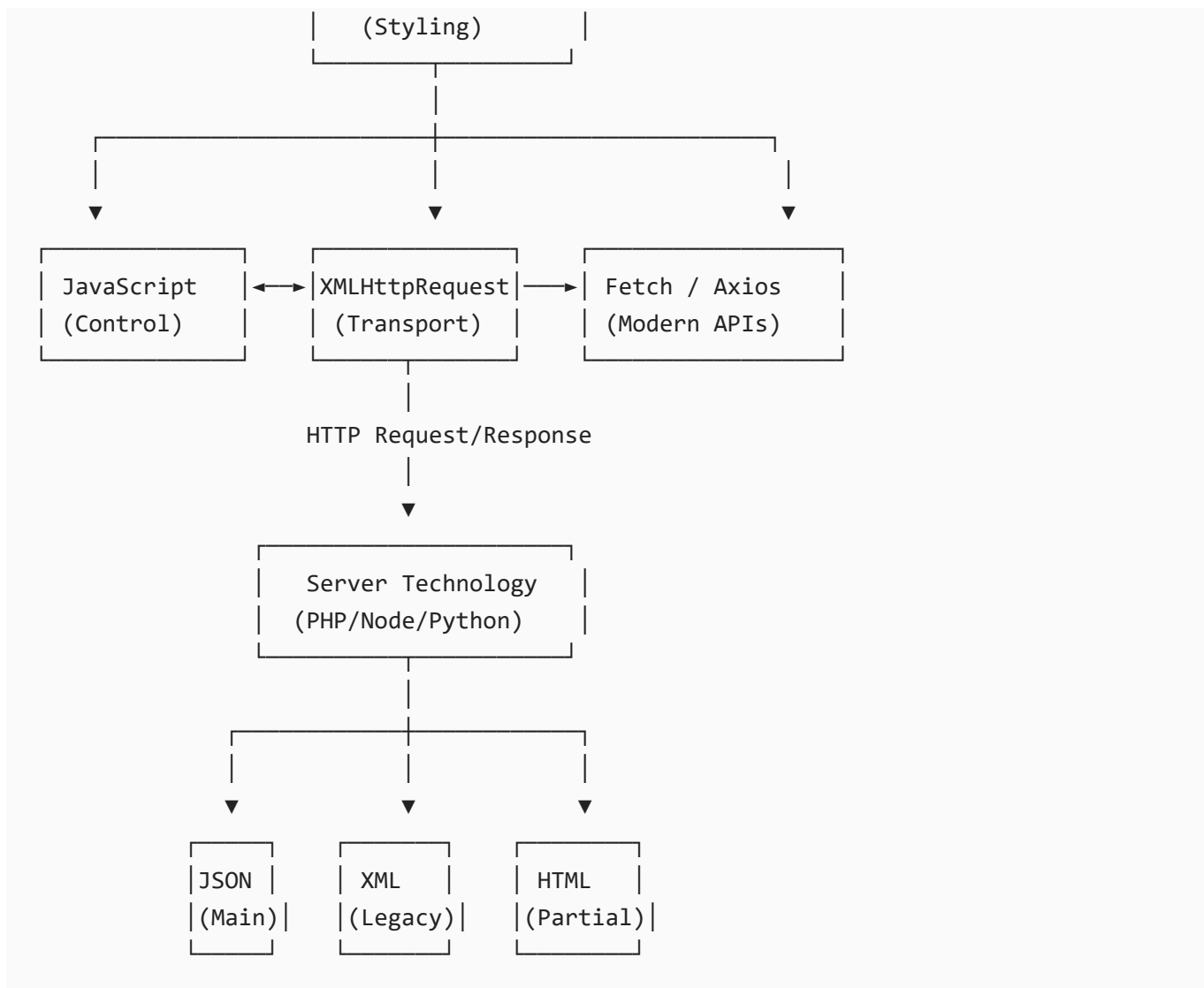
JSON Example:

```
{
  "status": "success",
  "data": {
    "users": [
      {"id": 1, "name": "Alice"},
      {"id": 2, "name": "Bob"}
    ]
  }
}
```

XML Example:

```
<?xml version="1.0"?>
<response>
  <status>success</status>
  <data>
    <user id="1">
      <name>Alice</name>
    </user>
  </data>
</response>
```





These three technologies (client-side, server-side, and data formats) work together seamlessly to create the smooth, responsive experience that defines modern web applications.

Explain MongoDB Data Types Along with Syntax

MongoDB stores data in BSON format.

Common Data Types

Data Type	Description	Example
String	Text data	"name": "Rahul"
Integer	Numeric value	"age": 21
Boolean	True/False	"status": true
Double	Decimal value	"marks": 85.5

Data Type	Description	Example
Array	Multiple values	"skills":["C","Java"]
Object	Embedded document	"address":{"city":"Mumbai"}
Date	Date value	new Date()
Null	Empty value	"middleName":null

Example Document

```
db.student.insertOne({
  name: "Amit",
  age: 22,
  marks: 88.5,
  active: true,
  skills: ["Java", "MongoDB"],
  address: {
    city: "Pune"
  },
  joinDate: new Date()
});
```

Advantages

- Flexible schema
- Supports complex data
- Easy nested document storage

What are the Features of MongoDB?

- **Document-Oriented Storage** - Uses JSON-like documents (BSON)
- **Schema-less Design** - Documents in same collection can have different fields
- **Indexing** - Supports single, compound, geospatial, and text indexes
- **Replication** - Replica sets with automatic failover
- **Sharding** - Horizontal scaling across distributed clusters
- **Aggregation Pipeline** - Powerful data processing framework
- **Ad-hoc Queries** - Query by field, range, regex, etc.
- **ACID Transactions** (v4.0+) - Multi-document transaction support
- **GridFS** - Store and retrieve large files (>16MB)
- **TTL Indexes** - Automatic data expiration

Discuss Default Databases in MongoDB: local, admin, and config

Database	Purpose
admin	Authentication, authorization, and system settings
local	Replica set operations (never replicated)
config	Sharded cluster metadata

admin Database:

```
// Create admin user
use admin
db.createUser({
  user: "adminUser",
  pwd: "securePass",
  roles: ["root"]
})
// List all users
db.getUsers()
```

local Database:

```
use local
// Shows replica set oplog
db.oplog.rs.find().limit(5)
// Shows replica set configuration
db.system.replset.findOne()
```

config Database (Sharded Clusters):

```
use config
// View shards
db.shards.find()
// View chunk distribution
db.chunks.find().pretty()
```

Note: local database is NOT replicated; admin and config are replicated only in certain configurations.

Explain MongoDB CRUD Operations with Example

CRUD stands for **Create, Read, Update, Delete** - the four basic database operations.

CREATE Operations:

Method	What it does	Syntax
<code>insertOne()</code>	Inserts a single document into a collection	<code>db.collection.insertOne(document)</code>
<code>insertMany()</code>	Inserts multiple documents into a collection	<code>db.collection.insertMany([doc1, doc2, ...])</code>

Examples:

```
// insertOne() examples
db.users.insertOne({ name: "John", age: 25, email: "john@email.com" })
db.products.insertOne({ productId: "P001", name: "Laptop", price: 999.99 })

// insertMany() examples
db.users.insertMany([
  { name: "Alice", age: 30 },
  { name: "Bob", age: 28 },
  { name: "Charlie", age: 35 }
])
```

READ Operations:

Method	What it does	Syntax
<code>find()</code>	Retrieves multiple documents matching a query	<code>db.collection.find(query, projection)</code>
<code>findOne()</code>	Retrieves a single document matching a query	<code>db.collection.findOne(query, projection)</code>

Examples:

```
// find() examples
db.users.find() // All documents
db.users.find({ age: { $gt: 25 } }) // Age greater than 25
db.users.find({}, { name: 1, age: 1, _id: 0 }) // Only name and age fields

// findOne() examples
```

```
db.users.findOne({ name: "John" })           // First John found
db.users.findOne({ email: "alice@email.com" }) // Find by email
```

UPDATE Operations:

Method	What it does	Syntax
updateOne()	Updates a single document matching the filter	db.collection.updateOne(filter, update, options)
updateMany()	Updates all documents matching the filter	db.collection.updateMany(filter, update, options)
replaceOne()	Replaces an entire document (keeps _id)	db.collection.replaceOne(filter, replacement)

Examples:

```
// updateOne() examples
db.users.updateOne({ name: "John" }, { $set: { age: 26 } })
db.users.updateOne({ email: "john@email.com" }, { $inc: { age: 1 } })

// updateMany() examples
db.users.updateMany({ status: "inactive" }, { $set: { status: "active" } })
db.users.updateMany({ age: { $lt: 30 } }, { $set: { category: "young" } })

// replaceOne() examples
db.users.replaceOne({ name: "John" }, { name: "John Smith", email:
"john.smith@email.com", age: 27 })
```

DELETE Operations

Method	What it does	Syntax
deleteOne()	Deletes a single document matching the filter	db.collection.deleteOne(filter)
deleteMany()	Deletes all documents matching the filter	db.collection.deleteMany(filter)

Examples:

```
// deleteOne() examples
db.users.deleteOne({ name: "John" })
db.users.deleteOne({ _id: ObjectId("507f1f77bcf86cd799439011") })
```

```
// deleteMany() examples
db.users.deleteMany({ status: "inactive" })
db.users.deleteMany({ age: { $lt: 18 } }) // Delete all minors
db.users.deleteMany({}) // Delete ALL documents
```

Operators:

Operator	What it does	Example
\$set	Sets the value of a field	{ \$set: { age: 30 } }
\$inc	Increments a field by a value	{ \$inc: { age: 1 } }
\$unset	Removes a field	{ \$unset: { tempField: "" } }
\$push	Adds an element to an array	{ \$push: { tags: "new" } }

Query Operators for Read Operations:

Operator	What it does	Example
\$gt	Greater than	{ age: { \$gt: 25 } }
\$lt	Less than	{ age: { \$lt: 30 } }
\$eq	Equal to	{ status: { \$eq: "active" } }
\$or	Logical OR	{ \$or: [{ age: 20 }, { age: 30 }] }
\$and	Logical AND	{ \$and: [{ age: 25 }, { status: "active" }] }

Explain Accessing and Manipulating Database Commands in MongoDB

MongoDB provides a rich set of commands and methods for accessing and manipulating databases. Commands are executed through the `db.runCommand()` method or directly using shell helpers.

Database-Level Commands

```
// Show all databases
show dbs

// Switch to/create a database
```

```
use myDatabase

// Check current database
db

// Drop current database
db.dropDatabase()
```

Collection Manipulation

```
// Create collection explicitly
db.createCollection("users")

// Show all collections
show collections

// Drop collection
db.users.drop()
```

CRUD Operations (Accessing & Manipulating)

Create (Insert)

```
// Insert one document
db.users.insertOne({
  name: "Alice",
  age: 30,
  email: "alice@example.com"
})

// Insert multiple documents
db.users.insertMany([
  { name: "Bob", age: 25 },
  { name: "Charlie", age: 35 }
])
```

Read (Access)

```
// Find all documents
db.users.find()

// Find with filter
db.users.find({ age: { $gt: 25 } })
```

```
// Find one document
db.users.findOne({ name: "Alice" })

// Projection (select specific fields)
db.users.find({}, { name: 1, age: 1, _id: 0 })
```

Update (Manipulate)

```
// Update one document
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 31 } }
)

// Update multiple documents
db.users.updateMany(
  { age: { $lt: 30 } },
  { $inc: { age: 1 } }
)

// Replace entire document
db.users.replaceOne(
  { name: "Bob" },
  { name: "Robert", age: 26 }
)
```

Delete

```
// Delete one document
db.users.deleteOne({ name: "Charlie" })

// Delete multiple documents
db.users.deleteMany({ age: { $lt: 20 } })
```

Query Operators for Advanced Access

Operator	Purpose	Example
\$eq	Equal	{ age: { \$eq: 30 } }
\$gt	Greater than	{ age: { \$gt: 25 } }
\$in	In array	{ status: { \$in: ["active", "pending"] } }
\$regex	Pattern matching	{ name: { \$regex: /^A/ } }

Indexing for Faster Access

```
// Create index
db.users.createIndex({ email: 1 })

// List indexes
db.users.getIndexes()
```

These commands form the foundation of MongoDB data manipulation, enabling efficient database operations.

Explain the Concept of Collections and Documents in MongoDB with Examples

Term	Equivalent in SQL	Description
Database	Database	Container for collections
Collection	Table	Group of documents
Document	Row/Record	Single data record (BSON format)

1. Documents

A document is a JSON-like structure (actually BSON) containing field-value pairs.

Basic Document Example

```
// Simple document
{
  "name": "Alice",
  "age": 28,
  "city": "New York"
}
```

Nested Document Example

```
// Document with nested objects and arrays
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "firstName": "Bob",
  "lastName": "Smith",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "state": "NY",
    "zip": "10001"
  },
  "hobbies": ["reading", "golfing", "traveling"]
}
```

```
"contact": {
  "email": "bob@email.com",
  "phone": "555-1234"
},
"hobbies": ["reading", "swimming", "coding"],
"isActive": true
}
```

2. Collections

A collection is a group of documents. Collections don't enforce a schema — documents within a collection can have different fields.

Creating a Collection

```
// Explicit creation (optional - MongoDB creates on first insert)
db.createCollection("users")

// Collection is automatically created when you insert first document
db.users.insertOne({ name: "Alice", age: 28 })
```

3. Operations (CRUD Examples)

Create Operations

```
// Insert one document
db.users.insertOne({
  name: "John Doe",
  email: "john@example.com",
  age: 30
})

// Insert multiple documents
db.users.insertMany([
  { name: "Alice", age: 25, city: "Boston" },
  { name: "Bob", age: 32, city: "Chicago" },
  { name: "Charlie", age: 28, city: "Denver" }
])
```

Read Operations

```
// Find all documents
db.users.find()
```



```
// Find with condition
db.users.find({ age: { $gt: 25 } })

// Find one document
db.users.findOne({ name: "Alice" })
```

Update Operations

```
// Update one document
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26, city: "Seattle" } }
)

// Update multiple documents
db.users.updateMany(
  { city: "Boston" },
  { $set: { state: "MA" } }
)

// Replace entire document
db.users.replaceOne(
  { name: "Bob" },
  { name: "Robert", age: 33, city: "Chicago" }
)
```

Delete Operations

```
// Delete one document
db.users.deleteOne({ name: "Charlie" })

// Delete multiple documents
db.users.deleteMany({ age: { $lt: 18 } })
```

Explain the Working of MongoDB and Applications of MongoDB

How MongoDB Works

MongoDB is a **NoSQL document database** that stores data in flexible, JSON-like documents (BSON format). Here's its working mechanism:

1. Core Architecture Components

- **Database:** Container for collections (like a schema in SQL)
- **Collection:** Group of documents (like a table in SQL, but schema-less)
- **Document:** Basic unit of data (JSON-like structure with key-value pairs)
- **Field:** Key-value pair within a document

2. Data Storage Format (BSON)

MongoDB stores data in **BSON** (Binary JSON), which extends JSON with additional data types like:

- Date, ObjectId, Binary data, Decimal128
- Arrays and nested documents

Example document:

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "name": "John Doe",
  "age": 29,
  "email": "john@example.com",
  "address": {
    "city": "New York",
    "zip": "10001"
  }
}
```

3. Indexing Mechanism

- Uses **B-Tree indexes** for efficient queries
- Supports single-field, compound, multikey (arrays), geospatial, and text indexes
- Automatically indexes the `_id` field

4. Sharding (Horizontal Scaling)

Data is distributed across multiple servers using:

- **Shard key** to partition data
- **Config servers** to maintain metadata
- **Mongos routers** to route queries to appropriate shards

5. Replication (High Availability)

- **Replica sets** with one primary and multiple secondary nodes

- Primary handles all writes; secondaries replicate data via oplog
- Automatic failover if primary goes down

6. Query Processing

- Supports CRUD operations, aggregations, text search, geospatial queries
- Uses **aggregation pipeline** for complex data transformations
- Supports ACID transactions (multi-document)

7. Storage Engines

- **WiredTiger** (default): Document-level concurrency, compression, encryption
- **In-Memory** (for caching/lookup scenarios)

Applications of MongoDB

1. Content Management Systems (CMS)

- Blogs, news sites, e-commerce catalogs
- **Why:** Flexible schema accommodates varying content structures (articles, videos, user comments)

2. Real-Time Analytics and Big Data

- Clickstream analysis, IoT sensor data, user behavior tracking
- **Why:** High write throughput and ability to query semi-structured time-series data

3. Mobile Apps (Backend)

- User profiles, push notifications, social feeds
- **Why:** JSON data format aligns with mobile app data structures; fast for read-heavy workloads

4. Product Catalogs & Inventory Management

- E-commerce platforms (Amazon, eBay-style)
- **Why:** Products have different attributes (electronics vs clothing); schema-less design handles variations easily

6. Gaming Leaderboards & User Data

- Game profiles, achievements, in-game purchases
- **Why:** High-speed reads/writes and ability to store player inventories as arrays

7. Log Management

- Application logs, server logs, audit trails
- **Why:** Easy ingestion of semi-structured log data; efficient time-based queries

8. Internet of Things (IoT)

- Device metadata, sensor readings, telemetry
- **Why:** Handles diverse device data formats and high-frequency writes

9. Customer Data Platforms (CDP)

- Unified customer profiles from multiple sources
- **Why:** Documents can store varied attributes and evolve as new data sources added